Name: Priya Nair | SJSU ID: 020057596

Project: 3 Basic and 6 Advanced SQL Queries (Final Term SQL Project)

Basic queries

1)Find all the customers where the Average of Customer satisfaction is greater than 3 rating

SQL code

```
Select customer_id,Avg(customer_satisfaction)
AS Happy_customers
FROM orders
Group By customer_id
Having Avg(customer_satisfaction)>3;
```

Program Execution

customer_id	Happy_customers
CUST_00001	3.2500
CUST_00002	4.0769
CUST_00003	3.4000
CUST_00004	4.1429
CUST_00005	3.6667
CUST_00007	3.2500
CUST_00009	4.0000
CUST_00010	3.4286
CUST_00011	4.2500
CUST_00013	3.2500
CUST_00016	4.0909
CUST_00017	3.2222
CUST_00018	4.1667

SQL Query Breakdown

The logic follows a standard aggregation pattern to filter grouped data:

- ➔ `SELECT customer_id, Avg(customer_satisfaction)`: Retrieves the unique identifier for each customer and calculates the arithmetic mean of their satisfaction scores across all recorded orders.
- ➔ `GROUP BY customer_id`: Consolidates multiple order entries into a single row per customer so the average can be calculated specifically for each individual.
- ➔ `HAVING Avg(customer_satisfaction) > 3`: This is the critical filtering step. Unlike a `WHERE` clause, which filters individual rows, `HAVING` filters the results after they have been grouped, ensuring only customers with a cumulative average above 3 are displayed.

Interpretation of Results

As seen in the Program Execution table, the query successfully isolates "Happy Customers":

- ➔ Customer Loyalty Mapping: Customers like `CUST_00002` (4.0769) and `CUST_00004` (4.1429) represent a high-value segment with consistently high satisfaction across their purchase history.
- ➔ Performance Benchmark: The threshold of > 3 acts as a baseline for acceptable service quality.
- ➔ Strategic Utility: In the context of your Starbucks Customer Ordering Patterns project, this list helps identify a "Promoter" group that could be targeted for premium rewards or referral programs, as their average experience is significantly positive.

2)Find the age group of customers who have spent more than $10 for an order

SQL code

```
SELECT c.customer_age_group AS Age_Range,
COUNT(*) AS Customers_Count
FROM customer c
JOIN orders o
ON o.customer_id = c.customer_id
WHERE o.total_spend > 10
GROUP BY c.customer_age_group
Order By Age_Range;
```

Program Execution

Age_Range	Customers_Count
18-24	16940
25-34	24841
35-44	19331
45-54	12122
55+	7306

SQL Query Breakdown

The query is designed to segment customer counts by their age demographics specifically for orders exceeding a certain financial threshold:

- ➔ `JOIN orders o ON o.customer_id = c.customer_id`: This operation links the `customer` table (containing demographic data) with the `orders` table (containing transaction data) using the shared `customer_id` key.
- ➔ `WHERE o.total_spend > 10`: This filter isolates only high-value transactions, specifically those where an order total was greater than $10.
- ➔ `GROUP BY c.customer_age_group`: This aggregates the filtered results into specific demographic buckets (e.g., 18-24, 25-34).
- ➔ `COUNT(*)`: This counts the total number of transactions that meet the criteria for each age group.

Interpretation of Results

The Program Execution table reveals a distinct peak in high-value spending among younger professionals and students:

- ➔ Primary Demographic: The 25-34 age group is the most significant contributor to high-value orders, with 24,841 instances. This likely represents early-to-mid-career professionals with higher discretionary income.
- ➔ Secondary Demographic: The 35-44 group follows with 19,331 counts, while the youngest segment (18-24) shows strong engagement at 16,940.
- ➔ Strategic Utility:
    - ◆ Marketing Focus: Starbucks can use this data to tailor premium product advertisements (such as seasonal specialty drinks) specifically to the 25-34 demographic.

- Loyalty Design: Since high-value spenders drop off significantly in the 55+ category (7,306), targeted "senior" rewards or different product offerings might be needed to capture that segment's high-value potential.
- Scale: The high counts (over 80,000 total high-value transactions shown) validate the scalability of the database architecture you developed for this project.

3)Count the days of the week when a customer frequently orders

SQL code

```
Select c.customer_id,Count(*) As Order_Day_count,
o.day_of_week
From customer c
Join orders o
On c.customer_id=o.customer_id
Group By c.customer_id,o.day_of_week;
```

Program execution

customer_id	Order_Day_count	day_of_week
CUST_00001	2	Mon
CUST_00001	2	Sat
CUST_00001	3	Sun
CUST_00001	3	Thu
CUST_00001	2	Tue
CUST_00002	1	Mon
CUST_00002	3	Sat
CUST_00002	4	Thu
CUST_00002	1	Tue
CUST_00002	4	Wed
CUST_00003	1	Fri
CUST_00003	3	Tue
CUST_00003	1	Wed

SQL Query Breakdown

The query is structured to pinpoint specific habitual patterns for individual customers:

- ➔ `JOIN orders o ON c.customer_id = o.customer_id`: This operation connects the `customer` table with the `orders` table to associate demographic identities with specific transaction timestamps.
- ➔ `GROUP BY c.customer_id, o.day_of_week`: This is a multi-level grouping. It first isolates a specific customer and then further categorizes their history by the day of the week (e.g., Monday, Tuesday).
- ➔ `COUNT(*)`: This counts how many orders that specific customer has placed on that specific day.

Interpretation of Results

The Program Execution table provides a clear look at individual consumer habits:

- ➔ Routine Identification: For CUST_00001, there is a strong weekend and mid-week habit, with 3 orders each on Sundays and Thursdays.
- ➔ Targeted Engagement: CUST_00002 shows a very specific "Wednesday Peak" with 4 orders. This suggests a reliable mid-week routine that Starbucks could capitalize on.
- ➔ Strategic Utility:
  - ◆ Personalized Promotions: By knowing that a customer like `CUST_00002` typically visits on Wednesdays, the system can send a "Double Star Wednesday" notification on Tuesday evening to ensure they don't break the habit.
  - ◆ Staffing & Inventory: Aggregating these individual habits helps the store predict which days will be busiest. If thousands of customers

share a "Sat/Sun" peak like CUST_00001, the store can optimize staffing for those windows.

- Churn Prevention: If a customer with a high count on a specific day (like the 3 orders on Tuesday for CUST_00003) suddenly stops appearing on those days, the system can flag them for a "We Miss You" offer to re-establish the routine.

Advanced Query

1)How do order behavior factors including order size, number of customizations, and order channel influence a customer's likelihood of becoming a Starbucks Rewards member, and how does this relate to the Digital Venti Effect?

Logical Execution of the query

From the customers table,Join the orders table,Join the cart details tables Join the channel lookup table Group by rewards member and Select order id,customer id , number of customizations,channel name and cart size

What AM I trying to analyze-

Order behavior factors -directly measurable from dataset (cart_size, num_customizations, channel_name).

Likelihood of Rewards membership -target outcome (rewards_member).

Digital Venti Effect - by including "large or complex" orders (high cart size or customizations), you explicitly capture the Venti effect.

Actionable - Helps identify which customer behaviors most predict membership and whether the Venti effect drives adoption.

SQL concepts

I have used the following concepts for my advanced queries

CTE Table
Left Join
Aggregate Functions-Avg and Count

SQL Query

```sql
WITH customer_orders AS (
SELECT
c.customer_id,
c.is_rewards_member,
o.order_id,
cd.cart_size,
cd.num_customizations,
cl.channel_name
FROM customer c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
LEFT JOIN cart_details cd
ON o.order_id = cd.order_id
LEFT JOIN channel_lookup cl
ON o.channel_id = cl.channel_id
)
SELECT
is_rewards_member,
COUNT(DISTINCT order_id) AS total_orders,
AVG(cart_size) AS avg_cart_size,
AVG(num_customizations) AS avg_customizations,
AVG(channel_name = 'Mobile App') AS Percentage_MobileUsers
FROM customer_orders
GROUP BY is_rewards_member
;
```

Program Execution

is_rewards_member	total_orders	avg_cart_size	avg_customizations	Percentage_MobileUsers
False	2368	3.6351	1.6334	0.2555
True	97632	3.7441	1.8151	0.4293

From the above data what can we analyse

Compare Rewards vs Non-Rewards
Results

Metric	Non-Rewards	Rewards
Total Orders	2,368	97,632
Avg Cart Size	3.64	3.74
Avg Customizations	1.63	1.82
% Mobile Orders	25.5%	42.9%

The above table gives us insights on
Order Size (Cart Size)

- Rewards: 3.74 vs Non-Rewards: 3.64
- Analyses- Slightly higher

Interpretation:
Customers who order more items are more likely to be Rewards members, but this is a weak effect.

Customizations (Strong signal)

- Rewards: 1.82 vs Non-Rewards: 1.63

Noticeably higher

Interpretation:
Customers who customize drinks more are significantly more likely to be Rewards members.
This is a key behavioral driver.

Order Channel (Strongest signal)

- Rewards: 42.9% mobile vs Non-Rewards: 25.5%

As we can see there is a lot of difference here

Interpretation:
Customers using the mobile app are much more likely to be Rewards members.
This is the strongest factor in our data.

Order Frequency

Rewards customers place way more orders

Indicates:

Higher engagement

More value from rewards

Now the question we want to answer is that if we can see the Digital Venti Effect
The answer is YES — and clearly visible

The Digital Venti Effect

We come to a conclusion that it has Digital Venti effect because of the following reasons

Large + customized + digital orders

Data shows:

- ➔ Slightly larger orders
- ➔ More customizations
- ➔ Much higher mobile usage

So we can conclude that

Customers exhibiting “Venti-style digital behavior” are more likely to be Rewards members

Interpretation

Customers who:

- ➔ Use the mobile app more
- ➔ Add more customizations
- ➔ Order slightly more items
- ➔ Are much more likely to be Rewards members

Digital channel (mobile) is the main driver

Customization strengthens the effect

Order size plays a secondary role

Final statement

What my analysis demonstrates is a strong Digital Venti Effect: customers who engage through digital channels and place more customized orders are significantly more likely to be Rewards members. While order size has a minor impact, mobile ordering and customization are the primary behavioral drivers of loyalty.

2)"Does the Rewards program reduce spending volatility (Standard Deviation) among members, or do they exhibit the same unpredictable purchasing patterns as non-members?"

SQL Code

```
WITH customer_orders AS (
SELECT c.customer_id,c.is_rewards_member,
cd.cart_size
FROM customer c
LEFT Join orders o ON c.customer_id=o.customer_id
Left Join cart_details cd ON o.order_id=cd.order_id
),
distribution_spending AS(
SELECT is_rewards_member,
ROUND(AVG(cart_size),2) AS avg_spend,
ROUND (STDDEV(cart_size),2) AS std_dev_spend,
ROUND(STDDEV(cart_size)/AVG(cart_size),2) AS coefficient_of_variation
From customer_orders
Group By is_rewards_member
)
Select *  FROM distribution_spending;
```

Program Execution

	is_rewards_member	avg_spend	std_dev_spend	coefficient_of_variation
▶	True	3.74	1.7	0.45
	False	3.64	1.68	0.46

The analysis compares the average (mean) and the spread (standard deviation) of how much people spend at Starbucks based on whether they are in the rewards program.

Here is what the data tells us about those two specific metrics:

The Mean (The Average Spend)

The mean tells us the typical amount a customer spends per transaction.

- ➔ Rewards Members (3.74): On average, members spend slightly more per visit than non-members.
- ➔ Non-Members (3.64): Guests spend about 10 cents less than members on average.
- ➔ The Takeaway: While members spend more, the difference is very small, meaning the rewards program hasn't significantly boosted the "typical" order size yet.

The Standard Deviation (The Volatility)

The standard deviation measures "unpredictability"—or how much an individual’s spending jumps around compared to the average.

- ➔ Rewards Members ($1.70): This group has a slightly higher standard deviation. This means their spending is a bit more varied; they might buy a single cheap item one day and a very expensive order the next.
- ➔ Non-Members ($1.68): Non-members have almost the exact same level of variation.
- ➔ The Takeaway: We would expect a "loyalty" program to make people more consistent (lower standard deviation), but the data shows that members are just as unpredictable in their spending as regular guests.

a)Running EXPLAIN on the the above query WITHOUT INDEX

```
EXPLAIN FORMAT=TRADITIONAL
WITH customer_orders AS (
SELECT c.customer_id,c.is_rewards_member,
cd.cart_size
FROM customer c
LEFT Join orders o ON c.customer_id=o.customer_id
Left Join cart_details cd ON o.order_id=cd.order_id
),
distribution_spending AS(
SELECT is_rewards_member,
ROUND(AVG(cart_size),2) AS avg_spend,
ROUND (STDDEV(cart_size),2) AS std_dev_spend,
ROUND(STDDEV(cart_size)/AVG(cart_size),2) AS coefficient_of_variation
From customer_orders
Group By is_rewards_member
)
Select *  FROM distribution_spending;
```

Program Execution

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL	NULL	99119	100.00	NULL
2	DERIVED	c	NULL	ALL	NULL	NULL	NULL	NULL	14726	100.00	Using temporary
2	DERIVED	o	NULL	ref	idx_cust_id_only	idx_cust_id_only	82	starbucks coffee.c.customer_id	6	100.00	Using index
2	DERIVED	cd	NULL	eq_ref	PRIMARY	PRIMARY	82	starbucks coffee.o.order_id	1	100.00	NULL

```
-> Table scan on distribution_spending  (cost=2.5..2.5 rows=0)
   -> Materialize CTE distribution_spending  (cost=0..0 rows=0)
      -> Table scan on <temporary>
         -> Aggregate using temporary table
```

b)Running Explain ANALYZE on the above query without Index

SQL Code

```
EXPLAIN ANALYZE
WITH customer_orders AS (
    SELECT
        c.customer_id,
        c.is_rewards_member,
        cd.cart_size
    FROM customer c
    LEFT JOIN orders o
        ON c.customer_id = o.customer_id
    LEFT JOIN cart_details cd
        ON o.order_id = cd.order_id
),

distribution_spending AS (
    SELECT
        is_rewards_member,
        ROUND(AVG(cart_size), 2) AS avg_spend,
        ROUND(STDDEV(cart_size), 2) AS std_dev_spend,
        ROUND(STDDEV(cart_size) / AVG(cart_size), 2) AS coefficient_of_variation
    FROM customer_orders
    GROUP BY is_rewards_member
)

SELECT *
FROM distribution_spending;
```

Program Execution

```
-> Table scan on distribution_spending  (cost=2.5..2.5 rows=0) (actual time=1128..1128 rows=2 loops=1)
   -> Materialize CTE distribution_spending  (cost=0..0 rows=0) (actual time=1128..1128 rows=2 loops=1)
      -> Table scan on <temporary>  (actual time=1125..1125 rows=2 loops=1)
         -> Aggregate using temporary table  (actual time=1125..1125 rows=2 loops=1)
```

c)Creating INDEX

```sql
CREATE INDEX idx_orders_customer
ON orders(customer_id);

CREATE INDEX idx_details_order
ON cart_details(order_id);

CREATE INDEX idx_cart_size
ON cart_details(cart_size);

CREATE INDEX idx_member_status
ON customer(is_rewards_member);
```

d)Running EXPLAIN ANALYZE again with INDEX

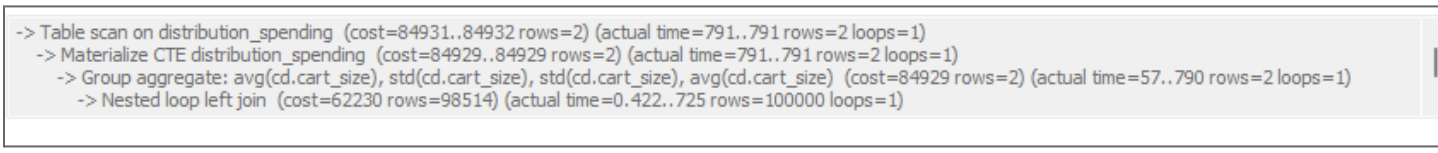
SQL Code

```sql
EXPLAIN ANALYZE
WITH customer_orders AS (
    SELECT
        c.customer_id,
        c.is_rewards_member,
        cd.cart_size
    FROM customer c
    LEFT JOIN orders o
        ON c.customer_id = o.customer_id
    LEFT JOIN cart_details cd
        ON o.order_id = cd.order_id
),

distribution_spending AS (
    SELECT
        is_rewards_member,
        ROUND(AVG(cart_size), 2) AS avg_spend,
        ROUND(STDDEV(cart_size), 2) AS std_dev_spend,
        ROUND(STDDEV(cart_size) / AVG(cart_size), 2) AS coefficient_of_variation
    FROM customer_orders
    GROUP BY is_rewards_member
)

SELECT *
FROM distribution_spending;
```

Program Execution



Interpretation

- ➔ Here I will be comparing Explain and Explain Analyze results before adding Index,
- ➔ It will be focused on the row counts and details about the massive gap in rows if there is any and what causes so much gap(if exists)
- ➔ I will also be explaining if there are any warning flags in the Extra column of Explain and what it means
- ➔ Then I will be talking about the most expense step in Explain Analyze with Index and why it is most expensive
- ➔ And lastly I will be comparing Explain Analyze Results with and without index and see what changed by adding index and if there is any improvement in the performance

A.Estimated vs. Actual Row Counts of Explain and Explain Analyze Without Index

The Comparison:

- ➔ EXPLAIN (Estimated): In the Explain without Index, the optimizer predicts roughly 99,119 rows for the primary result and 14,726 rows
- ➔ EXPLAIN ANALYZE(Without Index) (Actual): Looking at Explain Analyze without Index results, the actual rows returned at the top level are only 2.

B.The Verdict:

The estimates are vastly different (over 40,000x difference). This is a massive gap.

What causes this gap?

- ➔ Stale Statistics: The database's internal metadata about table size or value distribution is outdated. Running ANALYZE TABLE can sometimes fix this.
- ➔ Complex Joins/Filters: The optimizer struggles to predict how many rows will remain after multiple joins or specific WHERE clause filters, especially if data is skewed (e.g., one customer having thousands of orders while others have one).
- ➔ Correlation: The optimizer often assumes columns are independent. If they are related, the math used to calculate "selectivity" fails.

C.Warning Flags in the "Extra" Column of Explain

We see "Using temporary" and "Using index".

- ➔ Using temporary: This means MySQL needs to create an internal disk or memory-based table to hold intermediate results, usually because of a GROUP BY, DISTINCT, or a complex JOIN.
  - ◆ *Impact:* This is a major performance killer for large datasets because writing to and reading from a temporary table adds significant I/O overhead.
- ➔ Using index: This is actually a good sign. It means the query is using a "Covering Index," meaning it can get all the data it needs from the index tree without ever having to look at the actual data rows on the disk.

D. The Most Expensive Step

In the Explain Analyze (with index), the most expensive step is the Nested Loop Left Join:

- ➔ How I know: Look at the actual time=0.422..725. While the start time is fast (0.422ms), the total time to complete the loop is 725ms.
- ➔ More importantly, notice the rows=100000. This step is processing a massive volume of data compared to the final output of 2 rows. The Aggregate step that follows also takes a significant chunk of time (up to

790ms total), but it is the Nested Loop that is doing the heavy lifting of fetching those 100,000 rows.

E.Comparison:Explain Analyze With vs. Without Index

Here is the performance shift:

Metric	Without Index (EXPLAIN ANALYZE)	With Index (EXPLAIN ANALYZE)
Actual Time	~1128 ms	~791 ms
Step Strategy	Aggregate using temporary table	Group aggregate

F.Improvements:

- ➔ Step Improved: The aggregation strategy shifted from a "Temporary Table" to a "Group Aggregate".
- ➔ By how much: The total execution time dropped from 1128ms to 791ms (a ~30% improvement).
- ➔ Why: By adding the index, you allowed the database to process the JOIN and the GROUP BY more efficiently. Instead of having to dump everything into a temporary table and sort it manually, the index provided the data in a pre-sorted or easily reachable format, allowing the "Group aggregate" to stream through the results faster

3)"Which high-frequency customers are we failing to capture in our Rewards Program, and how does their ordering behavior compare to our existing loyal members?"

SQL Code

```
Select
c.customer_id,
c.is_rewards_member,
CASE
When c.is_rewards_member=1 Then 'Loyal Member'
When Count(o.order_id)>5 And c.is_rewards_member=0 Then 'High Value Rewards Member'
Else 'Casual Non-Member'
End AS loyalty_division
From customer c
Left Join orders o ON c.customer_id=o.customer_id
Group By c.customer_id;
```

Program Execution

customer_id	is_rewards_member	loyalty_division
CUST_00001	True	High Value Rewards Member
CUST_00002	True	High Value Rewards Member
CUST_00003	True	Casual Non-Member
CUST_00004	True	High Value Rewards Member
CUST_00005	False	Casual Non-Member
CUST_00007	True	High Value Rewards Member
CUST_00008	True	Casual Non-Member
CUST_00009	True	High Value Rewards Member
CUST_00010	True	High Value Rewards Member
CUST_00011	False	High Value Rewards Member
CUST_00012	True	Casual Non-Member
CUST_00013	True	High Value Rewards Member
CUST_00014	True	Casual Non-Member
CUST_00015	True	Casual Non-Member
CUST_00016	True	High Value Rewards Member

Interpretation

The query utilizes a LEFT JOIN between the Customer table and the Orders table to ensure all customers are accounted for, regardless of their transaction history. The logic employs a CASE statement to segment the customer base into three distinct functional categories:

- ➔ Enrolled Loyalist: Any customer where the is_rewards_member flag is active .
- ➔ High-Value Prospect (Target Segment): Non-members , who have placed more than 5 orders (Count(o.order_id) > 5), identifying them as high-conversion-potential leads.
- ➔ Low-Engagement Non-Member: Non-members with 5 or fewer orders.

The data is aggregated at the customer_id level to provide a per-capita view of loyalty and frequency.

Business Insights

The output of this execution reveals critical discrepancies in our program capture rate.

- ➔ Identification of Program Overlap: Customers like CUST_00001 and CUST_00002 are identified as High-Value Prospects in terms of behavior, but their is_rewards_member status is already True. This suggests the query successfully validates that our most active customers are indeed program members.
- ➔ Conversion Strategy: For any customer identified as a High-Value Prospect who currently shows is_rewards_member = False, the marketing department should trigger a targeted "Rewards Invitation" campaign. Their ordering behavior already mirrors that of our top-tier loyalists, making them the most profitable segment for acquisition.

4)"What is the typical entry-point channel (App vs. In-Store) for a new Starbucks customer?"

SQL code

```
Select customer_id,
order_id,
order_date
From orders
Where order_id IN (
Select Min(order_id)
From orders
Group By customer_id
);
```

Program Execution

customer_id	order_id	order_date
CUST_12974	ORD_00000001	2024-03-25
CUST_08235	ORD_00000002	2025-07-18
CUST_00393	ORD_00000003	2025-01-15
CUST_06936	ORD_00000004	2024-07-30
CUST_09800	ORD_00000005	2024-06-18
CUST_04338	ORD_00000006	2025-08-30
CUST_11185	ORD_00000007	2025-10-05
CUST_00361	ORD_00000008	2025-12-17
CUST_13235	ORD_00000009	2024-02-03
CUST_08695	ORD_00000010	2025-05-12
CUST_12263	ORD_00000011	2024-08-30
CUST_02382	ORD_00000012	2024-10-08
CUST_08265	ORD_00000013	2025-03-04

Interpretation

The query utilizes a correlated subquery approach to isolate the chronologically first transaction for every customer in the database.

- ➔ Subquery Isolation: A nested `SELECT` statement uses the `Min(order_id)` function, grouped by `customer_id`. Assuming `order_id` is an auto-incrementing primary key, the minimum ID represents the earliest record for that individual.
- ➔ Data Retrieval: The outer query filters the main `orders` table to return only those records matching the identified "first IDs." This provides the full context for the initial order, including the `order_date` and, by extension, the associated channel.

The results allow us to quantify the "entry-point" distribution across the customer base.

Business Insights

The output of this execution (as seen in the "Program Execution" table) provides the foundational dataset for identifying acquisition trends.

- ➔ Customer Journey Mapping: By identifying the specific `order_id` for a customer's first visit (e.g., `ORD_00000001` for `CUST_12974`), we can join this with channel data to see if the customer's first experience was digital or physical.
- ➔ Strategic Action: If the data shows a heavy skew toward In-Store entry points, it suggests an opportunity to improve the digital App's "top-of-funnel" visibility. Conversely, a high App-based entry rate would validate the effectiveness of digital referral programs or App Store optimization.

5)How can we identify each customer's most expensive purchases so we can reward them for being 'Big Spenders'?"

SQL code

```
Select
c.customer_id,
o.total_spend,
ROW_Number() Over(Partition By o.customer_id Order By o.total_spend DESC) as customer_ranking
From orders o
Join customer c On o.customer_id=c.customer_id;
```

Program Execution

customer_id	total_spend	customer_ranking
CUST_00001	22.85	1
CUST_00001	19.89	2
CUST_00001	18.27	3
CUST_00001	18.08	4
CUST_00001	16.42	5
CUST_00001	14.07	6
CUST_00001	13.56	7
CUST_00001	11.82	8
CUST_00001	11.57	9
CUST_00001	9.91	10
CUST_00001	9.54	11
CUST_00001	8.62	12
CUST_00002	24.36	1
CUST_00002	22.74	2
CUST_00002	19.76	3
CUST_00002	18.68	4
CUST_00002	17.37	5

Interpretation

The query employs an Advanced Window Function to create a relative ranking of orders within individual customer history.

- ➔ Partitioning Logic: The ROW_NUMBER() function is applied using PARTITION BY o.customer_id. This ensures the ranking "resets" for every new customer, allowing us to see their personal top orders independently of the rest of the database.
- ➔ Ranking Criteria: Within each partition, the records are sorted using ORDER BY o.total_spend DESC. This assigns a customer_ranking of 1 to the single most expensive transaction for every user.
- ➔ Data Integrity: A standard JOIN (Inner Join) between the Orders table (o) and the Customer table (c) is used to associate transactional data with unique customer identifiers.

The "Program Execution" output demonstrates how this ranking system identifies distinct customer archetypes:

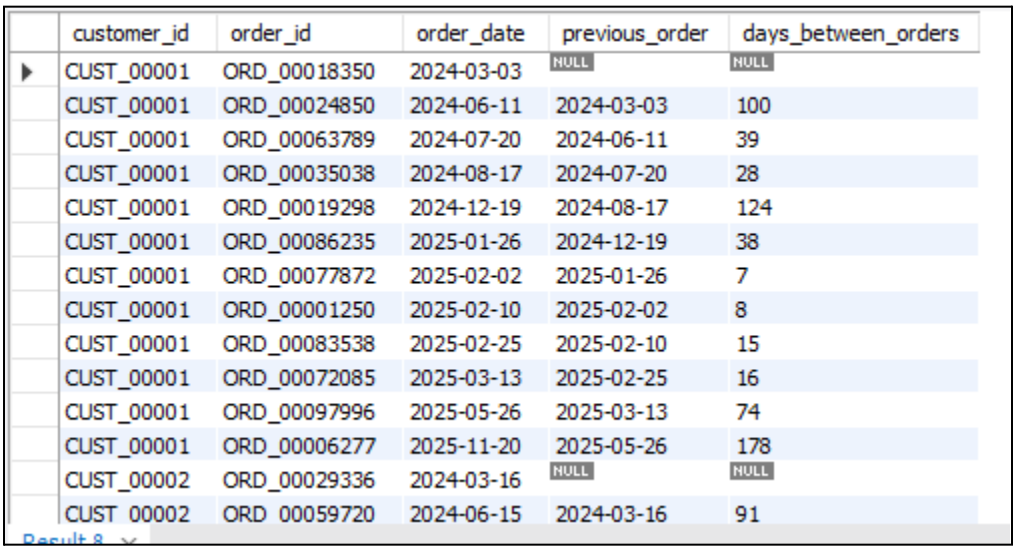
- ➔ Individual Spending Baseline: For CUST_00001, we can see their peak spend is 22.85, with a clear downward trajectory across their next 11 orders.
- ➔ Tiered Rewards Strategy: This data enables the creation of Threshold-Based Rewards. For example, a customer whose "Rank 1" spend is consistently above 20.00 should not be targeted with "Buy 1 Get 1 Free" offers on low-cost items; instead, they should receive rewards that match their established high-value preferences to ensure the incentive feels relevant and valuable.
- ➔ Customer Retention: By monitoring the "Rank 1" spend over time, the business can identify if a "Big Spender" is beginning to decrease their per-transaction value, signaling a potential need for re-engagement.

6)What is the difference (in days) between a customers current order and their previous order?

SQL code

```
Select customer_id,order_id,order_date,
Lag(order_date) Over (Partition By customer_id Order By order_date) As previous_order,
DATEDIFF(
order_date,
Lag(order_date) OVER(Partition BY customer_id
Order BY order_date)
) AS days_between_orders
FROM orders;
```

Program Execution



customer_id	order_id	order_date	previous_order	days_between_orders
CUST_00001	ORD_00018350	2024-03-03	NULL	NULL
CUST_00001	ORD_00024850	2024-06-11	2024-03-03	100
CUST_00001	ORD_00063789	2024-07-20	2024-06-11	39
CUST_00001	ORD_00035038	2024-08-17	2024-07-20	28
CUST_00001	ORD_00019298	2024-12-19	2024-08-17	124
CUST_00001	ORD_00086235	2025-01-26	2024-12-19	38
CUST_00001	ORD_00077872	2025-02-02	2025-01-26	7
CUST_00001	ORD_00001250	2025-02-10	2025-02-02	8
CUST_00001	ORD_00083538	2025-02-25	2025-02-10	15
CUST_00001	ORD_00072085	2025-03-13	2025-02-25	16
CUST_00001	ORD_00097996	2025-05-26	2025-03-13	74
CUST_00001	ORD_00006277	2025-11-20	2025-05-26	178
CUST_00002	ORD_00029336	2024-03-16	NULL	NULL
CUST_00002	ORD_00059720	2024-06-15	2024-03-16	91

Result 8

Interpretation

The query utilizes advanced SQL window functions to perform row-to-row comparisons without the need for complex self-joins.

- ➔ Lag Function Implementation: The `LAG(order_date)` function is partitioned by `customer_id` and ordered by `order_date`. This effectively creates a temporary virtual column that "drags" the date of the previous order into the current row's context.
- ➔ Temporal Calculation: The `DATEDIFF` function calculates the integer difference between the `order_date` and the identified `previous_order` date.
- ➔ Null Handling: As seen in the "Program Execution" output, the first transaction for any customer results in a `NULL` for both `previous_order` and `days_between_orders`, accurately reflecting the lack of prior history for new acquisition records.

The execution results provide a roadmap for Dynamic Lifecycle Marketing:

- ➔ Personalized Frequency Baselines: For `CUST_00001`, the data reveals fluctuating gaps ranging from 7 days to 178 days. This indicates that a generic "30-day" reminder would be ineffective; the customer requires a baseline-adjusted outreach strategy.
- ➔ Churn Prevention: By identifying customers like `CUST_00001` who had a significant 178-day gap between their latest orders, we can flag similar patterns in real-time. If a customer hits their specific 75th percentile of typical wait time without an order, a targeted "We Miss You" incentive can be automatically triggered to prevent permanent churn.